

About This Phase

Phase 1 taught you to think in terms of data and the functions that act on it, kept as two separate things: a dictionary here, a function that processes it over there. Phase 2 changes that. Object-oriented programming bundles data and the behavior that belongs with it into a single unit, a class, and it is how the vast majority of real, professional Python code, including every major library you'll meet from Phase 3 onward, is actually organized. By the end of these nine lessons, you'll be able to design your own classes, build them into families with inheritance, and understand exactly what's happening whenever you write `object.method()`, something you've already been doing since Lesson 1.7's string methods, without yet knowing precisely why it worked.

Every lesson follows the same structure as Phase 1: Why This Matters, Learning Objectives, the lesson itself, Common Mistakes to Avoid, and a Lesson Project.

Contents

Lesson 2.1: Classes & Objects	2
Lesson 2.2: Instance Methods & <code>self</code>	7
Lesson 2.3: Class & Static Methods	12
Lesson 2.4: Inheritance and <code>super()</code>	16
Lesson 2.5: Encapsulation & Property Decorators	21
Lesson 2.6: Magic Methods	26
Lesson 2.7: Decorators	32
Lesson 2.8: Abstract Base Classes	37
Lesson 2.9: Dataclasses	42
Phase 2 Capstone Project	47

Lesson 2.1: Classes & Objects

Why This Matters

Since Lesson 1.7, you've been calling methods on things: `"hello".upper()`, `my_list.append(3)`, `my_dict.get("key")`. Every one of those values, the string, the list, the dictionary, is an object, and `str`, `list`, and `dict` are all classes: blueprints that define what kind of data an object holds and what it can do. This lesson is where you stop only using classes Python already built for you, and start writing your own, which is how you'll model everything from a bank account to a video game character for the rest of this course.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain the difference between a class and an object, precisely
- Define a class and create objects (instances) from it
- Use `__init__` to give every object its own starting data
- Distinguish instance attributes from class attributes, and know when to use each

Classes and Objects: The Core Idea

A class is a blueprint: it describes what a kind of thing has and what it can do, without being any specific one of those things itself. An object (also called an instance) is one actual thing built from that blueprint. This distinction is exactly the same one that separates a cookie cutter from the cookies it makes: the cutter defines the shape, but you can't eat the cutter, and every cookie stamped from it is its own separate cookie, even though they all share the same shape.

```
1 class Dog:
2     pass
3
4 my_dog = Dog()
5 your_dog = Dog()
6
7 print(type(my_dog))      # <class '__main__.Dog'>
8 print(my_dog is your_dog) # False - two separate objects
```

`class Dog:` defines the blueprint, named `Dog` (class names conventionally use CapWords, unlike the `snake_case` from Lesson 1.3, which is how you'll immediately tell a class apart from a variable or function on sight). `pass` is a placeholder statement, meeting Python's requirement that a block can't be completely empty, standing in until there's real content inside. `Dog()` creates an object, an instance of `Dog`; the parentheses matter here exactly the way they mattered for calling a function since Lesson 1.11, since creating an object is, underneath everything, a kind of function call, one that builds a fresh, brand-new object and hands it back. `my_dog` and `your_dog` are two completely independent objects, exactly as `is` confirms, the same way two lists created separately in Lesson 1.8 were never the same list even if they happened to hold equal values; each call to `Dog()` allocates a new, distinct object in memory, so even two calls that looked identical still produce two different things, the same way two cookies cut from the same cutter are still two separate cookies, never one cookie in two places.

Giving Objects Data: `__init__`

An empty `Dog` isn't very useful; real objects carry their own data. `__init__` (short for "initialize") is a special method that runs automatically the moment an object is created, and it's where you set up whatever data that object should start with:

```
1 class Dog:
2     def __init__(self, name, breed):
3         self.name = name
4         self.breed = breed
5
6 my_dog = Dog("Rex", "Labrador")
7 your_dog = Dog("Bella", "Poodle")
8
9 print(my_dog.name)    # Rex
10 print(your_dog.name)  # Bella
11 print(my_dog.breed)   # Labrador
```

Walk through exactly what happens on `Dog("Rex", "Labrador")`. Python creates a new, blank object, then immediately calls `__init__` on it, automatically passing that new object in as `self`, with "Rex" and "Labrador" filling `name` and `breed`, matched by position exactly like any function call from Lesson 1.11. Inside `__init__`, `self.name = name` stores the value on the object itself, as an attribute, using the same dot syntax you've used to read attributes since Lesson 1.9's nested dictionaries, just now writing rather than reading. Once `__init__` finishes, `Dog("Rex", "Labrador")` hands back the fully set-up object, ready to use.

`self` is the parameter every instance method needs first, and it refers to the specific object the method was called on; `self.name` inside `__init__` means "store this on whichever object is currently being created," which is exactly why `my_dog.name` and `your_dog.name` can hold two completely different values despite coming from the exact same class and the exact same `__init__` code. Lesson 2.2 covers `self` in full depth; for now, treat it as a required first parameter that Python fills in automatically, never something you pass yourself.

Try It Yourself 2.1a: Define a class `Book` with an `__init__` that accepts `title`, `author`, and `pages`, storing all three as attributes. Create two different `Book` objects with different values, and print each one's title and pages to confirm they're independent.

Attributes: Reading, Setting, and Defaults

Once an object exists, you can read or change its attributes directly with dot notation, exactly like reading `my_dog.name` above:

```
1 my_dog = Dog("Rex", "Labrador")
2 print(my_dog.name)    # Rex
3
4 my_dog.name = "Max"    # attributes can be reassigned after creation
5 print(my_dog.name)    # Max
```

This should feel familiar: it's the exact same dot syntax Lesson 1.9 used for dictionary-like nested data, and precisely how you've been reading `.suffix` or `.stem` off a `Path` object since Lesson 1.13, without yet knowing `Path` was a class all along.

A parameter in `__init__` can have a default value, following the exact same rule from Lesson 1.11: parameters with defaults come after every parameter without one.

```
1 class Dog:
2     def __init__(self, name, breed, is_adopted=False):
3         self.name = name
4         self.breed = breed
5         self.is_adopted = is_adopted
6
```

```
7 rex = Dog("Rex", "Labrador")
8 print(rex.is_adopted) # False - used the default
9
10 bella = Dog("Bella", "Poodle", True)
11 print(bella.is_adopted) # True - explicitly overridden
```

Every rule from Lesson 1.11 about default parameter values, including never using a mutable value like a list as one directly, applies to `__init__` without any change, since `__init__` is simply a function, just one Python happens to call for you automatically.

Instance Attributes vs. Class Attributes

Every attribute you've seen so far, `self.name`, `self.breed`, is an instance attribute: it belongs to one specific object, and different objects hold different values for it. A class attribute is different: it's defined directly inside the class body, outside any method, and its value is shared by every instance of that class, unless a specific instance overrides it.

```
1 class Dog:
2     species = "Canis familiaris" # class attribute - shared by every Dog
3
4     def __init__(self, name, breed):
5         self.name = name # instance attribute - unique per Dog
6         self.breed = breed
7
8 rex = Dog("Rex", "Labrador")
9 bella = Dog("Bella", "Poodle")
10
11 print(rex.species) # Canis familiaris
12 print(bella.species) # Canis familiaris - the exact same value
13 print(rex.name) # Rex
14 print(bella.name) # Bella - different, as expected
```

`species` lives on the class itself, `Dog`, not on any individual dog; both `rex` and `bella` reach it through the same lookup, since Python checks an object's own instance attributes first, and falls back to the class's attributes if it doesn't find a match there. This makes a class attribute the right choice for data that's genuinely true of every instance alike, a species name, a fixed tax rate, a default configuration value, and the wrong choice for anything that should vary from one object to the next, which belongs in `__init__` as an instance attribute instead.

Reassigning through an instance, rather than through the class, creates a new instance attribute that shadows the class attribute for that one object alone, leaving every other instance, and the class itself, untouched:

```
1 rex.species = "Wolf (reclassified)"
2 print(rex.species) # Wolf (reclassified) - only rex changed
3 print(bella.species) # Canis familiaris - untouched
4 print(Dog.species) # Canis familiaris - the class itself is untouched too
```

This mirrors a distinction worth being precise about: `Dog.species` accesses the class attribute directly, on the blueprint itself, which is different from accessing it through any particular instance, exactly as `math.pi` in Lesson 1.15 was a value living on the `math` module itself, not on anything you created from it.

Try It Yourself 2.1b: Add a class attribute `bark_sound = "Woof!"` to your `Book` class from the earlier Try It Yourself (rename it something sensible, like a shared category = "Fiction"). Create two books, print the shared

class attribute through both, then change it on only one instance and confirm the other, and the class itself, are unaffected.

A Class Can Have Methods Too

Everything in this lesson so far has been about data; a class becomes genuinely useful once it also defines behavior, methods that act on that data. This is a brief preview, since Lesson 2.2 covers instance methods in full depth:

```
1 class Dog:
2     def __init__(self, name, breed):
3         self.name = name
4         self.breed = breed
5
6     def bark(self):
7         print(f"{self.name} says Woof!")
8
9 rex = Dog("Rex", "Labrador")
10 rex.bark() # Rex says Woof!
```

`bark(self)` is defined exactly like `__init__`, indented inside the class, taking `self` first; calling `rex.bark()` automatically passes `rex` in as `self`, which is how `bark` knows to use "Rex" specifically, rather than any other dog's name. This is the entire point of a class, stated plainly: data (`self.name`, `self.breed`) and the behavior that acts on it (`bark`) live together, in one unit, instead of a dictionary in one place and a separate function that happens to operate on it somewhere else, the way Phase 1 would have had to structure the exact same idea.

Common Mistakes to Avoid

- **Forgetting the parentheses when creating an object.** `my_dog = Dog` assigns the class itself to `my_dog`, not a new object; you need `Dog()` to actually call it and create an instance.
- **Forgetting `self` as the first parameter of `__init__`** (or any instance method). Without it, Python has no way to know which object's data to work with, and calling the method raises a `TypeError` about a mismatched number of arguments.
- **Confusing a class attribute with an instance attribute.** A class attribute is shared; changing it through one instance (rather than reassigning it, which shadows it instead) can unexpectedly affect every instance, especially dangerous with mutable class attributes like lists, an issue Lesson 2.2 revisits.
- **Using CapWords for a variable, or snake_case for a class.** It isn't enforced by Python, but breaking the convention makes code genuinely harder for anyone, including future you, to read at a glance.
- **Trying to access an instance attribute before `__init__` sets it.** If `__init__` only sets `self.name` under a certain condition, accessing `.name` on an object where that condition was false raises an `AttributeError`.

Lesson 2.1 Project: Contact Card

Write a program called `contact_card.py` that:

1. Defines a class `Contact` with an `__init__` accepting `name`, `phone`, and `email`, with `email` defaulting to `None` since not every contact will have one.
2. Adds a class attribute `contact_count` starting at 0.
3. Inside `__init__`, increments `Contact.contact_count` by 1 every time a new contact is created (accessing it through the class name, not `self`, since this is meant to track a running total across every instance).

4. Adds a method `display()` that prints the contact's details in a neatly formatted block, showing "No email on file" if email is `None`.
5. Creates at least three `Contact` objects with different data, calls `.display()` on each, and finally prints `Contact.contact_count` to show the running total across all of them.

This project deliberately uses a class attribute for genuinely shared, class-wide data (a running count) alongside instance attributes for data that's unique to each contact, the core distinction this lesson is built around.

Lesson Summary: Quick Reference

- A class is a blueprint; an object (or instance) is one specific thing built from it, with `ClassName()`
- `__init__(self, ...)` runs automatically on creation and sets up instance attributes with `self.attr = value`
- `self` always refers to the specific object a method was called on; it's passed automatically, never supplied by the caller
- Instance attributes (set in `__init__` via `self.`) are unique per object; class attributes (defined in the class body directly) are shared by every instance
- Reassigning an attribute through an instance creates or overrides it on that instance alone, leaving the class and other instances untouched

Lesson 2.2: Instance Methods & `self`

Why This Matters

Lesson 2.1 ended with a brief preview of `bark(self)`, a method, without fully explaining what `self` actually is or why every instance method needs it. That gap closes in this lesson. Instance methods are how objects actually do things, not just hold data, and understanding `self` precisely, rather than treating it as a magic word you type out of habit, is what lets you write methods that behave correctly instead of ones that merely happen to work by luck.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain precisely what `self` is and how Python fills it in automatically
- Write instance methods that read and modify an object's own attributes
- Call one method from inside another method on the same object
- Explain why `object.method()` and `Class.method(object)` do the exact same thing

What `self` Actually Is

Every method you define inside a class is really just a function, defined with `def`, indented one level inside the class body, following every rule from Lesson 1.11 about functions in general. The one difference is the first parameter, conventionally named `self`, which Python fills in automatically with whichever object the method was called on.

```
1 class Dog:
2     def __init__(self, name):
3         self.name = name
4
5     def bark(self):
6         print(f"{self.name} says Woof!")
7
8 rex = Dog("Rex")
9 rex.bark() # Rex says Woof!
```

Here is the precise mechanism, worth understanding exactly once so it never feels like magic again: `rex.bark()` is Python's shorthand for `Dog.bark(rex)`. Calling a method through an instance, with the dot, automatically passes that instance in as the method's first argument; `self` is simply the name given to that first parameter, receiving `rex` in this call. You can prove this to yourself directly:

```
1 rex.bark()          # the normal way
2 Dog.bark(rex)       # exactly equivalent - calling through the class, passing rex manually
```

Both lines print the exact same thing, because they're doing the exact same work; `object.method()` is purely convenient shorthand for `Class.method(object)`, not a fundamentally different kind of call. This is precisely why every instance method needs `self` listed first in its definition: Python is always going to supply the calling object as that first argument, whether or not you remember to declare a parameter to receive it.

`self` is a name, not a keyword; Python doesn't actually require that specific word, but every Python developer uses it, without exception, so a method that names its first parameter anything else is technically legal and immediately confusing to every other Python programmer who reads it, including future you. Treat `self` as if it were a fixed rule.

Methods That Use self to Read Data

The entire point of self is that it gives a method access to the specific object it was called on, which means a method can read that object's own attributes and compute something from them:

```
1 class Rectangle:
2     def __init__(self, width, height):
3         self.width = width
4         self.height = height
5
6     def area(self):
7         return self.width * self.height
8
9     def perimeter(self):
10        return 2 * (self.width + self.height)
11
12 box = Rectangle(4, 3)
13 print(box.area())      # 12
14 print(box.perimeter()) # 14
```

area(self) reads self.width and self.height, the exact values __init__ stored on box when it was created, and returns their product. Every instance method has this same shape available to it: self.attribute always means “the value of this attribute, on whichever object this method was actually called on,” which is exactly why calling .area() on a differently-sized rectangle gives a genuinely different, correct answer without area's own code changing at all. Create a second rectangle, box2 = Rectangle(10, 2), and box2.area() returns 20, not 12; the exact same line of code inside area, return self.width * self.height, produces a different answer purely because self is bound to a different object each time, with its own separate width and height.

Methods That Use self to Change Data

Methods aren't limited to reading and returning; they can also update an object's own attributes, permanently changing its state, exactly the way a list's .append() from Lesson 1.8 permanently changes the list it's called on:

```
1 class BankAccount:
2     def __init__(self, owner, balance=0):
3         self.owner = owner
4         self.balance = balance
5
6     def deposit(self, amount):
7         self.balance += amount
8         print(f"Deposited {amount}. New balance: {self.balance}")
9
10    def withdraw(self, amount):
11        if amount > self.balance:
12            print("Insufficient funds.")
13        else:
14            self.balance -= amount
15            print(f"Withdrew {amount}. New balance: {self.balance}")
16
17 account = BankAccount("Ada", 100)
18 account.deposit(50)    # Deposited 50. New balance: 150
19 account.withdraw(30)   # Withdrew 30. New balance: 120
20 account.withdraw(500)  # Insufficient funds.
```

`self.balance += amount` permanently modifies the balance stored on that specific account object; the change persists for as long as the object exists, exactly like reassigning any attribute directly would, since `+=` here is just shorthand for `self.balance = self.balance + amount`, the same augmented assignment rule from Lesson 1.4. This is the core, practical difference between an instance method that merely computes something (like `area()`) and one that changes the object's own state (like `deposit()`); both are completely ordinary uses of `self`, and a real class typically has methods of both kinds.

Try It Yourself 2.2a: Add a method `apply_interest(self, rate)` to the `BankAccount` class that increases `self.balance` by `balance * rate` (for example, a rate of 0.05 for 5%), printing the new balance afterward. Create an account, deposit some money, then apply interest, and confirm the balance updates correctly each step.

One Method Calling Another

A method can call other methods on the same object, and it does so exactly the way any code outside the class would, through `self`:

```
1 class BankAccount:
2     def __init__(self, owner, balance=0):
3         self.owner = owner
4         self.balance = balance
5
6     def deposit(self, amount):
7         self.balance += amount
8
9     def withdraw(self, amount):
10        if amount > self.balance:
11            print("Insufficient funds.")
12        else:
13            self.balance -= amount
14
15    def transfer_to(self, other_account, amount):
16        self.withdraw(amount)
17        other_account.deposit(amount)
18        print(f"Transferred {amount} from {self.owner} to {other_account.owner}.")
19
20 ada_account = BankAccount("Ada", 200)
21 alan_account = BankAccount("Alan", 50)
22
23 ada_account.transfer_to(alan_account, 75)
24 print(ada_account.balance)    # 125
25 print(alan_account.balance)  # 125
```

`self.withdraw(amount)` inside `transfer_to` calls the `withdraw` logic on the same account `transfer_to` itself was called on, reusing it rather than duplicating the balance-checking logic a second time. `other_account.deposit(amount)` is different, and worth noticing precisely: it calls a method on a completely different object, one passed in as an argument, exactly the same way you'd call a method on any object handed to a function. This pattern, an object's own method coordinating calls to its own other methods, and to methods on different objects it's been given, is how real classes build complex behavior out of small, focused pieces, rather than writing one enormous method that does everything at once.

Try It Yourself 2.2b: Add a method `summary(self)` to `BankAccount` that calls no other method, just returns a formatted string like `"Ada: $125"` using an f-string. Then add a method `print_summary(self)` that calls `self.summary()` and prints the result, demonstrating one method on an object calling another method on that same object.

Methods Can Take Any Argument Functions Can

Beyond `self`, an instance method accepts parameters exactly like any function from Lesson 1.11: positional, keyword, default values, `*args`, `**kwargs`, all of it, simply written after `self` in the parameter list.

```

1 class ShoppingCart:
2     def __init__(self):
3         self.items = []
4
5     def add_item(self, name, price, quantity=1):
6         self.items.append({"name": name, "price": price, "quantity": quantity})
7
8     def total(self):
9         return sum(item["price"] * item["quantity"] for item in self.items)
10
11 cart = ShoppingCart()
12 cart.add_item("Book", 12.99, quantity=2)
13 cart.add_item("Pen", 1.50)
14 print(cart.total()) # 27.48

```

`add_item` takes `self` first, as every instance method must, followed by ordinary parameters, one with a default; calling `cart.add_item("Book", 12.99, quantity=2)` supplies "Book" and 12.99 positionally and `quantity` by keyword, exactly following Lesson 1.11's rules, since `self` being filled in automatically doesn't change how the remaining parameters behave at all.

Common Mistakes to Avoid

- **Forgetting `self` in a method's parameter list.** `def bark():` inside a class, missing `self`, raises a `TypeError` the moment you call `my_dog.bark()`, since Python is still automatically passing the object in, and there's no parameter left to receive it.
- **Passing `self` explicitly when calling through an instance.** `rex.bark(rex)` passes `rex` twice, once automatically and once manually, and raises a `TypeError` about too many arguments; call it as plain `rex.bark()`.
- **Forgetting the `self.` prefix when reading or setting an attribute inside a method.** Writing just `balance += amount` instead of `self.balance += amount` creates or reads a local variable, entirely disconnected from the object's actual data, and raises a `NameError` if that local variable was never assigned first.
- **Trying to call a method on the class itself, without an instance, forgetting to pass one.** `Dog.bark()` raises a `TypeError`, since `bark` still expects a `self` argument that nothing supplied; either call it through an instance, or explicitly as `Dog.bark(rex)`.
- **Confusing a method that returns a value with one that only prints.** Exactly as in Lesson 1.11, a method that only `print()`s and never returns hands back `None` to anything trying to use its result, like `total = cart.total()` when `total()` only printed instead of returning.

Lesson 2.2 Project: Task Manager

Write a program called `task_manager.py` that:

1. Defines a class `TaskList` whose `__init__` sets up an empty list attribute to hold tasks.

2. Adds a method `add_task(self, description, priority="normal")` that appends a dictionary (with `description`, `priority`, and `done` starting as `False`) to the task list.
3. Adds a method `complete_task(self, description)` that finds a task by its description and sets its `done` to `True`, printing a message if no task with that description exists.
4. Adds a method `pending_count(self)` that returns how many tasks currently have `done` equal to `False`, using it from inside a `show_summary(self)` method that prints the total number of tasks and how many are still pending, calling `self.pending_count()` rather than recalculating it separately.
5. Creates one `TaskList`, adds at least four tasks with a mix of priorities, marks two of them complete, and calls `show_summary()` to display the final state.

This project deliberately requires methods that read data, methods that modify data, and one method calling another on the same object, the three core skills from this lesson.

Lesson Summary: Quick Reference

- An instance method is a function defined inside a class, with `self` as its required first parameter
- `object.method()` is shorthand for `Class.method(object)`; Python fills `self` in automatically either way
- `self.attribute` inside a method always refers to that attribute on the specific object the method was called on
- A method can read attributes to compute a return value, or modify attributes to change the object's state, or both
- One method can call another method on the same object through `self.other_method()`, or a method on a different object passed in as an argument

Lesson 2.3: Class & Static Methods

Why This Matters

Every method you've written so far has been an instance method, taking `self` and acting on one specific object's data. Not every method a class needs actually works with a specific instance, though: some belong conceptually to the class as a whole, and some don't need any object's data at all, just a logical home inside the class. Python provides two more kinds of method for exactly these cases, and knowing which of the three to reach for is a genuinely practical, everyday design skill.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain the difference between an instance method, a class method, and a static method
- Write a class method using `@classmethod` and `cls`, including the alternative-constructor pattern
- Write a static method using `@staticmethod`
- Decide which of the three kinds of method fits a given situation

A Quick Preview of Decorators

Both `@classmethod` and `@staticmethod` use `@` syntax, called a decorator, placed directly above a method definition. Lesson 2.7 covers decorators in full depth, including how to write your own; for this lesson, treat `@classmethod` and `@staticmethod` as built-in labels that tell Python “this method works differently from a normal instance method,” and focus on what each one actually changes.

Class Methods: Working With the Class Itself

A class method receives the class itself as its first argument, conventionally named `cls`, instead of receiving a specific instance as `self`. It's defined with `@classmethod` directly above it:

```
1 class Dog:
2     total_dogs = 0
3
4     def __init__(self, name):
5         self.name = name
6         Dog.total_dogs += 1
7
8     @classmethod
9     def get_total(cls):
10         return cls.total_dogs
11
12 rex = Dog("Rex")
13 bella = Dog("Bella")
14
15 print(Dog.get_total()) # 2 - called on the class itself, no instance needed
```

`cls` inside `get_total` refers to `Dog`, the class, exactly the way `self` inside an instance method refers to whichever object the method was called on. `cls.total_dogs` reads the class attribute from Lesson 2.1 directly, and `Dog.get_total()` calls it through the class itself, with no instance in sight, which makes sense here: counting every dog ever created is a fact about the `Dog` class as a whole, not about any one specific dog, so it has no real reason to need a `self`.

A class method can also be called through an instance, and Python still passes the class, not the instance, as `cls`:

```
1 print(rex.get_total()) # 2 - still works, cls is still Dog, not rex
```

This is worth knowing but rarely the way class methods are actually called in practice; calling through the class name directly, as `Dog.get_total()`, communicates the intent more clearly, since the method genuinely has nothing to do with any specific instance.

The Alternative Constructor Pattern

The single most common, genuinely practical use of `@classmethod` is building an alternative way to create an object, when the normal `__init__` doesn't fit every situation data might arrive in. Suppose `Dog` normally expects a name and a birth year, but you sometimes have an age instead:

```
1 from datetime import date
2
3 class Dog:
4     def __init__(self, name, birth_year):
5         self.name = name
6         self.birth_year = birth_year
7
8     @classmethod
9     def from_age(cls, name, age):
10        birth_year = date.today().year - age
11        return cls(name, birth_year)
12
13 rex = Dog("Rex", 2020) # the normal way
14 bella = Dog.from_age("Bella", 3) # the alternative way, computing birth_year first
15
16 print(rex.birth_year) # 2020
17 print(bella.birth_year) # 2023 (assuming the current year is 2026)
```

`from_age` does the extra work of converting an age into a birth year, and then calls `cls(name, birth_year)`, which is precisely `Dog(name, birth_year)`, since `cls` is `Dog` here, running the normal `__init__` exactly as if you'd called it directly. Writing `cls(...)` instead of hardcoding `Dog(...)` is a deliberate, important detail: it means `from_age` keeps working correctly even for a subclass of `Dog` (subclasses are covered fully in Lesson 2.4), automatically building an instance of whichever class it was actually called through, rather than always forcing a plain `Dog`.

This pattern, an alternative constructor named to describe what it builds from, is genuinely everywhere in real Python code; you'll recognize it immediately once you know to look for it. `dict.fromkeys(...)` and `datetime.fromtimestamp(...)` are both classmethods built on exactly this pattern, in the standard library itself.

Try It Yourself 2.3a: Add a class method `from_full_name(cls, full_name)` to a simple `Person` class (with an `__init__` taking `first_name` and `last_name`) that splits a single string like "Ada Lovelace" on the space (using `.split()` from Lesson 1.7) and calls `cls(first, last)` with the two pieces. Create one `Person` the normal way and one using `Person.from_full_name("Alan Turing")`, printing both.

Static Methods: Just Living in the Class

A static method takes neither `self` nor `cls`; it's an ordinary function that doesn't need any particular instance's or the class's own data, but is placed inside the class anyway because it's conceptually related, and grouping it there keeps related code organized together. It's defined with `@staticmethod`:

```

1 class TemperatureConverter:
2     @staticmethod
3     def celsius_to_fahrenheit(celsius):
4         return celsius * 9 / 5 + 32
5
6     @staticmethod
7     def fahrenheit_to_celsius(fahrenheit):
8         return (fahrenheit - 32) * 5 / 9
9
10 print(TemperatureConverter.celsius_to_fahrenheit(100)) # 212.0
11 print(TemperatureConverter.fahrenheit_to_celsius(32)) # 0.0

```

Neither method needs `self` (no specific `TemperatureConverter` instance's data is involved at all) or `cls` (no class-level data like `Dog.total_dogs` is involved either); each is a self-contained calculation that simply belongs, conceptually, with temperature conversion. You could write these as plain, standalone functions outside any class entirely, and they'd work identically; the reason to make them static methods instead is purely organizational, keeping every temperature-related tool grouped under one recognizable name, `TemperatureConverter`, rather than scattered as loose functions in a module.

Static methods are called through the class, exactly like class methods, and, like class methods, can also be called through an instance if one happens to exist, though that's rarely the more natural way to reach for them:

```

1 converter = TemperatureConverter()
2 print(converter.celsius_to_fahrenheit(0)) # 32.0 - works, but the class name is more
    idiomatic

```

Try It Yourself 2.3b: Add a static method `is_valid_priority(priority)` to a small `Task` class that returns `True` if `priority` is one of "low", "normal", or "high", and `False` otherwise, using the `in` keyword from Lesson 1.7. Call it directly through the class name with a few different values, with no `Task` object ever created.

Choosing the Right Kind of Method

Three kinds of method, three different jobs, and the choice comes down to one question: what data does the method actually need?

Method type	First parameter	Use when
Instance method	<code>self</code>	The method needs a specific object's own data, like <code>self.balance</code>
Class method	<code>cls</code>	The method needs the class itself, most often to build an object a different way (an alternative constructor) or to work with a class attribute
Static method	neither	The method needs no instance's or class's data at all, but belongs with the class conceptually

In practice, the overwhelming majority of methods you write will be ordinary instance methods, exactly like every one from Lesson 2.2; class methods and static methods are reached for specifically, for the narrower cases this lesson covers, not as a default choice.

Common Mistakes to Avoid

- **Using `self` in a method decorated with `@classmethod`.** A class method's first parameter receives the class, conventionally named `cls`; mixing up the name doesn't break anything technically, but it's confusing and against every real-world convention.
- **Hardcoding the class name inside a class method instead of using `cls`.** Writing `return Dog(name, birth_year)` inside `from_age` works for `Dog` itself, but breaks the alternative-constructor pattern for any subclass; `return cls(name, birth_year)` is the correct, flexible form.
- **Making every method a static method out of uncertainty.** If a method genuinely needs to read or change an object's own data, it should be a normal instance method with `self`; reaching for `@staticmethod` by default defeats the purpose of bundling data and behavior together in the first place.
- **Forgetting the decorator entirely and getting a confusing error.** A method meant to be a class method, but missing `@classmethod`, receives an actual instance as its first argument instead of the class, which usually surfaces as a confusing `AttributeError` once the method tries to use `cls` as if it were the class.
- **Calling a static or class method as if it required an instance first.** It doesn't; `TemperatureConverter.celsius_to_fahrenheit(100)` is the natural, idiomatic way to call it, with no object needed anywhere.

Lesson 2.3 Project: Pizza Order Builder

Write a program called `pizza_builder.py` that:

1. Defines a class `Pizza` with an `__init__` accepting size and a list of toppings, plus a class attribute `orders_placed` starting at 0, incremented inside `__init__` every time a pizza is created.
2. Adds class methods `margherita(cls)` and `pepperoni(cls)` that each act as alternative constructors, calling `cls(...)` with a preset size and topping list for that specific style of pizza.
3. Adds a class method `get_orders_placed(cls)` that returns `cls.orders_placed`.
4. Adds a static method `estimate_price(size, topping_count)` that returns a price using a simple formula of your choosing (for example, a base price for the size plus a fixed amount per topping), needing no instance or class data at all.
5. Creates a few pizzas, mixing the normal constructor with the alternative ones, prints each one's estimated price using the static method, and finally prints the total number of orders placed using the class method.

This project deliberately uses all three method types from this lesson for the purpose each is actually suited to.

Lesson Summary: Quick Reference

- Instance methods (`self`) act on one object's own data; this is the default, most common kind of method
- Class methods (`@classmethod, cls`) act on the class itself, most often as alternative constructors that build and return `cls(...)`
- Static methods (`@staticmethod, no automatic first parameter`) need no instance or class data, but belong with the class conceptually
- Always use `cls(...)`, not the hardcoded class name, inside a classmethod constructor, so subclasses build correctly too
- Choose based on what data the method actually needs: an object's, the class's, or neither

Lesson 2.4: Inheritance and `super()`

Why This Matters

Real programs often model several kinds of thing that are clearly related but not identical: a savings account and a checking account are both bank accounts, sharing most of their behavior, but each has something the other doesn't. Writing each one as a completely separate class means duplicating everything they share, and duplicated code is exactly the kind of thing that quietly drifts out of sync the moment you fix a bug in one copy and forget the other. Inheritance is how you write the shared part once and build specific variations on top of it.

Learning Objectives

By the end of this lesson, you will be able to:

- Create a subclass that inherits attributes and methods from a parent class
- Override a parent's method in a subclass, and extend it with `super()` instead of replacing it entirely
- Explain what `super().__init__()` does and why forgetting it causes real bugs
- Use `isinstance()` and `issubclass()` to check relationships between objects and classes

Creating a Subclass

A subclass (or child class) is built on top of an existing class (the parent, base, or superclass), automatically gaining every attribute and method the parent defines, without rewriting any of it:

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5     def eat(self):
6         print(f"{self.name} is eating.")
7
8     def make_sound(self):
9         print(f"{self.name} makes a sound.")
10
11 class Dog(Animal):
12     pass
13
14 rex = Dog("Rex")
15 rex.eat()           # Rex is eating. - inherited from Animal, unchanged
16 rex.make_sound()   # Rex makes a sound. - also inherited
```

`class Dog(Animal):` is the entire syntax for inheritance: the parent class's name goes in parentheses right after the child's name. `Dog` here defines nothing of its own at all, just `pass`, and yet `rex.eat()` and `rex.make_sound()` both work, because `Dog` automatically inherits every attribute and method `Animal` defines, including `__init__` itself; `Dog("Rex")` runs `Animal's __init__`, since `Dog` never defined its own. `isinstance(rex, Animal)` is `True` here too, alongside `isinstance(rex, Dog)`, since every `Dog` genuinely is an `Animal`, just a more specific kind of one.

Overriding a Method

A subclass can redefine a method it inherited, replacing the parent's version entirely for that subclass:

```
1 class Animal:
2     def __init__(self, name):
```

```

3         self.name = name
4
5     def make_sound(self):
6         print(f"{self.name} makes a sound.")
7
8 class Dog(Animal):
9     def make_sound(self):
10        print(f"{self.name} says Woof!")
11
12 class Cat(Animal):
13     def make_sound(self):
14        print(f"{self.name} says Meow!")
15
16 rex = Dog("Rex")
17 whiskers = Cat("Whiskers")
18
19 rex.make_sound()      # Rex says Woof!
20 whiskers.make_sound() # Whiskers says Meow!

```

Both Dog and Cat define their own `make_sound`, and Python always uses the most specific version available: calling `.make_sound()` on a Dog finds Dog's own `make_sound` first, and never even looks at Animal's version, which is exactly what overriding means. This is a genuinely powerful pattern, worth naming directly: the same method call, `.make_sound()`, produces different, appropriate behavior depending on the actual type of object it's called on, without any calling code needing to check what kind of animal it has first with an `if/elif` chain. This idea has a name, polymorphism, and it's one of the central reasons object-oriented code scales well as a program grows: adding a new kind of animal later means writing one new class, not modifying every place `make_sound` might get called.

Extending, Not Just Replacing: `super()`

Sometimes a subclass wants to add to what the parent's method already does, rather than replace it completely. `super()` gives you a way to call the parent's version of a method from inside the subclass's own overriding version:

```

1 class Animal:
2     def __init__(self, name, species):
3         self.name = name
4         self.species = species
5
6     def describe(self):
7         print(f"{self.name} is a {self.species}.")
8
9 class Dog(Animal):
10    def __init__(self, name, breed):
11        super().__init__(name, species="Dog")
12        self.breed = breed
13
14    def describe(self):
15        super().describe()
16        print(f"Specifically, a {self.breed}.")
17
18 rex = Dog("Rex", "Labrador")
19 rex.describe()
20 # Rex is a Dog.
21 # Specifically, a Labrador.

```

`super().__init__(name, species="Dog")` calls `Animal`'s own `__init__`, doing exactly what it would do on its own, setting `self.name` and `self.species`, before `Dog`'s `__init__` continues with its own extra line, `self.breed = breed`. Without that `super().__init__(...)` call, `self.name` and `self.species` would simply never get set at all, since defining `__init__` in `Dog` completely replaces `Animal`'s version, rather than running both; `super()` is precisely how you opt back into the parent's behavior, deliberately, on top of the subclass's own additions.

The same pattern applies to describe: `super().describe()` runs `Animal`'s original version first, printing the general description, and then `Dog`'s own `describe` adds one more, more specific line underneath. This is the precise difference between overriding a method that replaces the parent's behavior entirely (like `make_sound` in the previous section) and overriding one that extends it (like `describe` here); both are completely valid, and the choice depends purely on whether the parent's original behavior is still wanted, in addition to something new.

Try It Yourself 2.4a: Write a class `Vehicle` with an `__init__` taking `make` and `model`, and a method `info(self)` that prints them. Write a subclass `ElectricVehicle` whose `__init__` takes `make`, `model`, and `battery_range`, calling `super().__init__()` for the first two, and whose own `info(self)` calls `super().info()` before printing the battery range as an additional line.

Checking Types: `isinstance()` and `issubclass()`

`isinstance(object, Class)` answers “is this object an instance of this class, or any of its subclasses,” returning a plain `bool`, and you've already used it, without necessarily naming it as such, any time you've reasoned about a value's type:

```
1 rex = Dog("Rex", "Labrador")
2
3 print(isinstance(rex, Dog))      # True - rex is directly a Dog
4 print(isinstance(rex, Animal))  # True - a Dog is also an Animal, through inheritance
5 print(isinstance(rex, Cat))     # False - unrelated class
```

This is precisely why `isinstance(rex, Animal)` is `True`: inheritance means every `Dog` genuinely is a kind of `Animal`, not merely similar to one, and `isinstance()` reflects that real relationship rather than checking for an exact, literal type match. `issubclass(ClassA, ClassB)` asks the equivalent question about two classes directly, rather than an object and a class:

```
1 print(issubclass(Dog, Animal))  # True
2 print(issubclass(Animal, Dog))  # False - the relationship isn't symmetric
```

Every class in Python, including every one you define yourself, ultimately inherits from a common base class called `object`, whether you write it explicitly or not; `class Animal:` and `class Animal(object):` mean exactly the same thing. This is worth knowing mainly so nothing about `issubclass(Animal, object)` being `True` ever comes as a surprise later.

Multiple Levels of Inheritance

A subclass can itself be the parent of another subclass, building a genuine hierarchy, exactly the way `Dog` is a more specific `Animal`, and something could be a more specific kind of `Dog` still:

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5 class Dog(Animal):
6     def bark(self):
```

```

7         print(f"{self.name} says Woof!")
8
9     class ServiceDog(Dog):
10         def __init__(self, name, task):
11             super().__init__(name)
12             self.task = task
13
14         def perform_task(self):
15             print(f"{self.name} is performing: {self.task}")
16
17 rex = ServiceDog("Rex", "guiding")
18 rex.bark()           # Rex says Woof! - inherited from Dog
19 rex.perform_task()   # Rex is performing: guiding
20 print(isinstance(rex, Animal)) # True - through two levels of inheritance

```

ServiceDog inherits from Dog, which itself inherits from Animal; `super().__init__(name)` inside ServiceDog calls Dog's `__init__` specifically (since Dog doesn't define its own `__init__` here, that search continues automatically up to Animal's), and `rex.bark()` reaches all the way up to Dog's method, since ServiceDog never overrides it. Python always searches from the most specific class outward, through as many levels as a hierarchy actually has, stopping at the very first matching method or attribute it finds; this search order is called the method resolution order, and for the straightforward, single-parent hierarchies this course builds, it's simply "check this class, then its parent, then its parent's parent," and so on, exactly as this example traces through.

Try It Yourself 2.4b: Extend your Vehicle/ElectricVehicle classes from the earlier Try It Yourself with a further subclass `ElectricSUV(ElectricVehicle)`, adding a `cargo_capacity` attribute in its own `__init__` (calling `super().__init__()` for the rest), and confirm with `isinstance()` that an `ElectricSUV` object is an instance of all three classes: `ElectricSUV`, `ElectricVehicle`, and `Vehicle`.

Common Mistakes to Avoid

- **Forgetting `super().__init__()` in a subclass's own `__init__`.** Defining `__init__` in a subclass completely replaces the parent's, rather than running both automatically; skipping `super().__init__()` means whatever attributes the parent's `__init__` would have set never get set at all, usually surfacing later as a confusing `AttributeError`.
- **Confusing overriding (replacing) with extending (calling `super()` and adding more).** Both are valid, but they're different decisions; forgetting `super()` when you actually wanted the parent's behavior too silently loses it.
- **Writing `class Dog(Animal())`: with parentheses after the parent's name.** Inheritance syntax uses the class name itself, `class Dog(Animal):`, never an instance of it.
- **Assuming `issubclass()` is symmetric.** `issubclass(Dog, Animal)` being `True` does not mean `issubclass(Animal, Dog)` is also `True`; the relationship only goes one direction, from more specific to more general.
- **Overusing inheritance for classes that aren't genuinely a specialized version of each other.** Inheritance models an "is-a" relationship (a Dog is an Animal); if two classes merely happen to share a few methods without one truly being a kind of the other, composition (one class holding an instance of the other as an attribute) is usually the better, more flexible tool.

Lesson 2.4 Project: Employee Hierarchy

Write a program called `employee_hierarchy.py` that:

1. Defines a base class `Employee` with an `__init__` taking `name` and `base_salary`, and a method `calculate_pay(self)` that simply returns `self.base_salary`.
2. Defines a subclass `Manager(Employee)` whose `__init__` takes `name`, `base_salary`, and `team_size`, calling `super().__init__()` for the first two, and whose `calculate_pay(self)` calls `super().calculate_pay()` and adds a bonus of \$200 per team member on top.
3. Defines a subclass `Executive(Manager)` whose `__init__` takes everything `Manager` does plus `stock_options`, calling `super().__init__()` appropriately, and whose `calculate_pay(self)` calls `super().calculate_pay()` and adds the value of the stock options on top.
4. Creates one of each class, calls `.calculate_pay()` on all three, and prints the results, along with confirming with `isinstance()` that the `Executive` object is also an instance of `Manager` and `Employee`.

This project deliberately builds a three-level hierarchy, using `super()` at every level to extend, rather than replace, the pay calculation each class inherits.

Lesson Summary: Quick Reference

- `class Child(Parent):` creates a subclass that inherits every attribute and method from `Parent`
- A subclass can override a method entirely, or extend it with `super().method_name()` to keep the parent's behavior and add to it
- `super().__init__(...)` calls the parent's constructor; forgetting it means the parent's setup never runs
- `isinstance(obj, Class)` checks membership including subclasses; `issubclass(A, B)` checks the class relationship directly, in one direction only
- Inheritance can span multiple levels; Python always uses the most specific matching method or attribute it finds, searching outward from the object's own class

Lesson 2.5: Encapsulation & Property Decorators

Why This Matters

Every attribute you've written so far has been wide open: anyone with an object can set `account.balance = -500` directly, bypassing every rule `deposit()` and `withdraw()` were carefully written to enforce back in Lesson 2.2. Encapsulation is the practice of controlling access to an object's data, so that changes only happen through methods that can validate them, and Python's property decorators are how you add that control without changing how the attribute is actually used from the outside.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain Python's naming convention for non-public attributes, and what it does and doesn't enforce
- Use `@property` to expose a method as if it were a plain attribute
- Use a matching `@x.setter` to validate a value before it's stored
- Create a genuinely read-only computed attribute

Python's Approach to Privacy: Convention, Not Enforcement

Many languages let a class truly lock an attribute away from outside code, raising an error if anything outside the class tries to touch it directly. Python takes a different, deliberately more relaxed stance, often summarized as “we're all consenting adults here”: nothing is ever truly unreachable, but naming conventions clearly signal intent, and everyone is expected to respect them.

```
1 class BankAccount:
2     def __init__(self, owner, balance):
3         self.owner = owner
4         self._balance = balance # single underscore: "internal use, please don't touch
                                   directly"
5
6 account = BankAccount("Ada", 100)
7 print(account._balance) # 100 - this works; Python does NOT prevent it
8 account._balance = -500 # also works, and also a bad idea
```

A single leading underscore, `_balance`, is a convention meaning “this is an implementation detail; treat it as internal, even though I can't stop you from reaching in.” Nothing about Python actually blocks `account._balance` from being read or written directly; the underscore is a signal to other developers, not a lock. This differs sharply from the strict access control some other languages enforce, and it's a genuine, deliberate design choice in Python: trust developers to respect a clear signal, rather than build technical walls everywhere.

A double leading underscore, `__balance`, goes a step further, triggering something called name mangling: Python internally renames the attribute to include the class name, making it awkward, though still not impossible, to access accidentally from outside:

```
1 class BankAccount:
2     def __init__(self, owner, balance):
3         self.owner = owner
4         self.__balance = balance
5
6 account = BankAccount("Ada", 100)
7 print(account.__balance) # AttributeError - not found under this name
```

```
8 print(account._BankAccount__balance) # 100 - the real, mangled name, still technically
   reachable
```

Name mangling exists mainly to avoid accidental name clashes in inheritance hierarchies, covered more in Phase 3, rather than to provide genuine security; in everyday code, a single underscore is by far the more common convention, and this course uses it as the default going forward. Neither underscore style stops determined outside code from reaching in; the actual, reliable tool for controlling access, validating changes, and computing derived values is the property system covered in the rest of this lesson.

@property: Making a Method Look Like an Attribute

@property lets you define a method that callers access using plain attribute syntax, no parentheses, while the method's code still runs every single time:

```
1 class Circle:
2     def __init__(self, radius):
3         self._radius = radius
4
5     @property
6     def area(self):
7         return 3.14159 * self._radius ** 2
8
9 circle = Circle(5)
10 print(circle.area) # 78.53975 - no parentheses, even though area() is a method
```

circle.area looks exactly like reading a plain attribute, the same syntax you've used since Lesson 2.1, but it's actually calling the area method behind the scenes and computing a fresh result every time, from the object's current _radius. This is precisely the point: area was never really stored data at all; it's a value derived from _radius, and @property lets the calling code treat it exactly like a plain attribute anyway, without needing to know, or care, that a calculation is actually happening underneath.

Trying to call it like a normal method fails, which is worth seeing once so it's never a surprise:

```
1 print(circle.area()) # TypeError - a float isn't callable; area is accessed WITHOUT
   parentheses
```

@property genuinely changes how the method is called, not just how it looks; once decorated, it must always be accessed exactly like an attribute, never with a trailing ().

Adding a Setter: Validating on the Way In

A property defined with just @property, on its own, is read-only; trying to assign to it raises an AttributeError. Pairing it with a setter, using @propertyname.setter, lets you accept assignment too, while running your own validation code first:

```
1 class BankAccount:
2     def __init__(self, owner, balance):
3         self.owner = owner
4         self._balance = balance
5
6     @property
7     def balance(self):
8         return self._balance
```

```
9
10 @balance.setter
11 def balance(self, value):
12     if value < 0:
13         raise ValueError("Balance cannot be negative.")
14     self._balance = value
15
16 account = BankAccount("Ada", 100)
17 print(account.balance) # 100 - calls the getter
18
19 account.balance = 250 # calls the setter, value=250, passes validation
20 print(account.balance) # 250
21
22 account.balance = -50 # calls the setter, raises ValueError before storing anything
```

Notice the getter and setter share the exact same name, `balance`, which is required: `@balance.setter` specifically refers back to the property just defined immediately above it. `account.balance = 250` looks like a plain attribute assignment, but it's actually calling the setter method, passing 250 in as `value`; the setter validates it, using `raise ValueError(...)` from Lesson 1.14 to reject a bad value before it's ever stored, and only assigns to the real, underlying `self._balance` once the value passes. This is the entire point of this lesson, made concrete: outside code still writes the simple, familiar `account.balance = 250`, with no idea a validation check is even happening, while the class retains complete, genuine control over what actually gets stored, something a plain public attribute could never enforce.

Try It Yourself 2.5a: Write a class `Temperature` with an internal `_celsius` attribute, a `celsius` property with a getter and a setter, where the setter raises a `ValueError` if the value is below -273.15 (absolute zero). Create a `Temperature` object, set a valid value and print it, then try setting an invalid one and confirm the error is raised.

Read-Only Computed Properties

Not every property needs a setter; a property with only a getter is genuinely read-only, useful for values that should always be derived from other data, never set directly at all:

```
1 class Rectangle:
2     def __init__(self, width, height):
3         self.width = width
4         self.height = height
5
6     @property
7     def area(self):
8         return self.width * self.height
9
10 box = Rectangle(4, 3)
11 print(box.area) # 12 - always current, computed from width and height
12
13 box.area = 100 # AttributeError - area has no setter, it's genuinely read-only
```

This is a meaningfully better design than storing `area` as a plain attribute set once in `__init__`: if `width` or `height` changes later, a stored `area` would silently go stale, still reflecting the old dimensions, while a computed property is always correct, recalculated fresh from whatever `width` and `height` currently hold, every single time it's accessed. Reach for a read-only property, exactly like this one, whenever a value should always be derived, and never independently settable at all.

Try It Yourself 2.5b: Add a read-only property `fahrenheit` to your `Temperature` class from the earlier Try It Yourself, computing it from `self._celsius` using the conversion formula from Lesson 1.17 ($f = c * 9/5 + 32$), with no setter at all. Confirm it updates automatically after changing `.celsius` through its setter.

Why This Is Better Than a Plain Attribute From the Start

It's worth being explicit about why this lesson's approach is genuinely better than simply making `balance` a plain public attribute from the beginning, rather than treating properties as an arbitrary extra rule to memorize. A plain attribute offers no opportunity to validate, compute, or react to a change at all; a property looks identical from the outside, the exact same `object.attribute` and `object.attribute = value` syntax, but gives the class a place to enforce its own rules, computed values, or side effects, entirely invisibly to whoever is using the object. This is precisely why many real, professional Python classes start with plain attributes and convert specific ones to properties later, exactly when validation or computation genuinely becomes necessary: the calling code never has to change at all, since the syntax on the outside stays identical either way.

Common Mistakes to Avoid

- **Calling a property with parentheses.** `circle.area()` raises a `TypeError`, since `@property` makes it accessed exactly like a plain attribute, never with `()`.
- **Forgetting the setter needs the exact same method name as the getter.** `@balance.setter` must decorate a method also named `balance`; a mismatched name simply defines an unrelated, ordinary method instead of a working setter.
- **Assigning to a property that only has a getter.** This raises `AttributeError`, since there's no setter to handle the assignment; that's often the entire point, for a genuinely read-only computed value like `area`.
- **Believing a single or double underscore genuinely prevents outside access.** Neither one does; they're conventions (or, for double underscores, name mangling), not real enforcement. Validation that actually matters belongs in a property's setter, not in a naming convention alone.
- **Putting expensive or side-effect-heavy work inside a property getter.** Since a property is accessed with plain attribute syntax, callers reasonably expect it to be fast and side-effect free, exactly like reading any other attribute; save genuinely heavy work for an explicitly named method instead.

Lesson 2.5 Project: Validated Product

Write a program called `validated_product.py` that:

1. Defines a class `Product` with an `__init__` accepting `name`, storing `price` and `quantity` through their properties (defined next) rather than assigning to `_price/_quantity` directly, so validation runs even during construction.
2. Adds a `price` property whose setter raises `ValueError` if the value is negative.
3. Adds a `quantity` property whose setter raises `ValueError` if the value is negative, or isn't a whole number (check with `isinstance(value, int)` from Lesson 1.3).
4. Adds a read-only `total_value` property that returns `price * quantity`, with no setter, always freshly computed.
5. Creates a `Product`, prints its `total_value`, changes `quantity` through its property, and prints `total_value` again to confirm it updated automatically. Then attempts to set a negative price, catching the resulting `ValueError` with `try/except` from Lesson 1.14 and printing a friendly message instead of crashing.

Lesson Summary: Quick Reference

- A single leading underscore (`_name`) signals “internal, please don’t touch directly”; it’s convention only, not enforced
- `@property` turns a method into something accessed with plain attribute syntax, no parentheses
- `@propertyname.setter` adds validated assignment, using the exact same name as the getter
- A property with only a getter is genuinely read-only, ideal for values that should always be freshly computed, never stored stale
- Properties keep the outside syntax identical to a plain attribute while giving the class real control over validation and computation

Lesson 2.6: Magic Methods

Why This Matters

You've been using magic methods since Lesson 1.3 without a name for them: `len(my_list)`, `my_list[0]`, `"a" + "b"`, `5 == 5`, every one of these works because the underlying class defines a special method Python calls automatically behind the scenes. `__init__` itself, from Lesson 2.1, is one of them. This lesson shows you how to write your own, which is what lets a class you design print nicely, compare correctly, support `len()`, and even work with `+`, exactly like Python's own built-in types do.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what a magic (dunder) method is and why Python calls it automatically
- Implement `__str__` and `__repr__` to control how an object displays
- Implement `__eq__` and ordering methods to make objects comparable
- Implement `__len__` and `__getitem__` to make a custom class behave like a built-in collection
- Overload an operator like `+` with `__add__`

What a Magic Method Actually Is

A magic method (also called a dunder method, short for “double underscore”) is a method whose name starts and ends with two underscores, like `__init__` or `__len__`, that Python calls automatically in response to a specific operation, rather than something you ever call directly yourself with dot notation. You never write `my_list.__len__()`; you write `len(my_list)`, and Python translates that call into `__len__` for you, behind the scenes.

```
1 numbers = [1, 2, 3]
2 print(len(numbers))           # 3
3 print(numbers.__len__())      # 3 - the exact same thing, called directly; never do this
                               in real code
```

This is precisely why `__init__` is a magic method too: `Dog("Rex")` doesn't call `__init__` directly by name; creating an object with `ClassName(...)` is the operation, and Python calls `__init__` automatically as part of it, exactly the same pattern as `len(x)` calling `__len__` automatically. Every magic method in this lesson follows this identical shape: a normal-looking piece of Python syntax or a built-in function, on one side, and a specific dunder method your class can define, on the other, connected automatically.

Controlling How an Object Prints: `__str__` and `__repr__`

Without any magic methods defined, printing a custom object gives an unhelpful, generic result:

```
1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6 p = Point(3, 4)
7 print(p) # <__main__.Point object at 0x7f8a1c0b3d90>
```

That default output tells you the object's type and its location in memory, neither of which is useful for actually understanding what the object represents. `__str__` defines what `print()` and `str()` should show instead, a readable, human-friendly description:

```

1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def __str__(self):
7         return f"Point({self.x}, {self.y})"
8
9 p = Point(3, 4)
10 print(p)          # Point(3, 4) - calls __str__ automatically
11 print(str(p))     # Point(3, 4) - the exact same thing, explicitly

```

`__repr__` is a close relative, meant for a different audience: where `__str__` targets a human reading output, `__repr__` targets a developer debugging, and is meant, as a convention, to be unambiguous, ideally even valid Python that could recreate the object.

```

1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def __repr__(self):
7         return f"Point(x={self.x}, y={self.y})"
8
9 p = Point(3, 4)
10 print(p)          # Point(x=3, y=4) - __repr__ is used since __str__ isn't defined here
11 p                 # in an interactive session, typing just p also shows __repr__'s result

```

If a class defines `__repr__` but not `__str__`, `print()` falls back to using `__repr__` instead, which is exactly what happened above. If both are defined, `print()` and `str()` use `__str__` specifically, while things like an interactive session showing a returned value, or an object printed inside a list, use `__repr__`. As a practical habit: define `__repr__` on nearly every class you write, since it's invaluable while debugging, and add `__str__` as well specifically when you also want a cleaner, separate display meant for an actual end user.

Try It Yourself 2.6a: Add `__repr__` to a simple `Book` class (with `title` and `author` attributes) returning something like `"Book(title='Dune', author='Herbert')"`. Create a list of three `Book` objects and `print()` the list directly, noticing that each object inside it displays using `__repr__`.

Making Objects Comparable: `__eq__` and Ordering

Without any magic methods defined, `==` between two objects of your own class checks identity, whether they're literally the same object in memory, exactly the `is` behavior from Lesson 1.4, which is rarely what you actually want when comparing two objects that merely hold equal data:

```

1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5

```

```

6 p1 = Point(3, 4)
7 p2 = Point(3, 4)
8 print(p1 == p2) # False - different objects in memory, even though the data matches

```

`__eq__` defines what `==` should actually check for that class:

```

1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def __eq__(self, other):
7         return self.x == other.x and self.y == other.y
8
9 p1 = Point(3, 4)
10 p2 = Point(3, 4)
11 print(p1 == p2) # True - now comparing actual data, not identity

```

`__eq__(self, other)` receives the object on the right-hand side of `==` as `other`, and returns a `bool` deciding whether the two should count as equal; here, two points are considered equal precisely when both coordinates match, which is exactly the definition of “equal” that actually makes sense for a coordinate. Trace through the comparison directly: `p1 == p2` becomes `p1.__eq__(p2)`, exactly the same automatic translation every magic method in this lesson follows; inside that call, `self` is `p1` and `other` is `p2`, so `self.x == other.x` checks `3 == 3` (`True`) and `self.y == other.y` checks `4 == 4` (`True`), and combines them, from Lesson 1.4, into a single `True` result.

Ordering comparisons, `<`, `>`, `<=`, `>=`, each have their own corresponding magic method, `__lt__`, `__gt__`, `__le__`, `__ge__`, following the exact same pattern:

```

1 class Product:
2     def __init__(self, name, price):
3         self.name = name
4         self.price = price
5
6     def __lt__(self, other):
7         return self.price < other.price
8
9     def __repr__(self):
10        return f"Product({self.name}, {self.price})"
11
12 products = [Product("Pen", 1.50), Product("Book", 12.99), Product("Mug", 8.00)]
13 products.sort()
14 print(products)
15 # [Product(Pen, 1.5), Product(Mug, 8.0), Product(Book, 12.99)]

```

Defining just `__lt__` is enough to make `.sort()` and `sorted()` from Lesson 1.8 work directly on a list of your own objects, with no `key=` function needed at all; Python’s sorting only ever needs to know how to compare two items with “less than,” and `__lt__` supplies exactly that.

Try It Yourself 2.6b: Add `__eq__` to your `Book` class, considering two books equal if their title and author both match. Then add `__lt__` comparing by title alphabetically (string comparison already works this way, from Lesson 1.4), and confirm `sorted()` correctly alphabetizes a list of `Book` objects.

Making a Class Behave Like a Collection: `__len__` and `__getitem__`

`__len__` lets `len()` work on your own class, and `__getitem__` lets square-bracket indexing, `obj[0]`, work on it too, both from Lesson 1.7 and 1.8's sequence syntax, now available on a class you write yourself:

```

1 class Playlist:
2     def __init__(self, songs):
3         self.songs = songs
4
5     def __len__(self):
6         return len(self.songs)
7
8     def __getitem__(self, index):
9         return self.songs[index]
10
11 playlist = Playlist(["Song A", "Song B", "Song C"])
12
13 print(len(playlist))      # 3 - calls __len__
14 print(playlist[0])       # Song A - calls __getitem__
15 print(playlist[-1])      # Song C - negative indexing works too, since it's delegated to
16                           # a list
17
18 for song in playlist:    # __getitem__ alone is enough to make a for loop work
19     print(song)

```

`__getitem__(self, index)` receives whatever index was used in the square brackets and decides what to return; here, it simply delegates to `self.songs[index]`, letting the underlying list handle the actual indexing logic, including negative indices, entirely for free. This delegation pattern, a custom class wrapping a built-in collection and forwarding the interesting operations to it, is genuinely common and worth recognizing: it lets `Playlist` present a clean, purpose-built interface (methods named for what a playlist actually does, covered further in a moment) while reusing a list's already-correct behavior underneath, rather than reimplementing indexing from scratch. Notably, defining `__getitem__` alone is enough to make a plain `for` loop work over a `Playlist` object directly, since Python's `for` falls back to repeatedly calling `__getitem__` with increasing indices, starting at 0, until it hits an `IndexError`, if no more specialized iteration support is defined.

Operator Overloading: `__add__` and Friends

Every arithmetic operator from Lesson 1.4 has a corresponding magic method, letting you define what `+`, `-`, `*`, and others should mean for your own class:

```

1 class Money:
2     def __init__(self, dollars):
3         self.dollars = dollars
4
5     def __add__(self, other):
6         return Money(self.dollars + other.dollars)
7
8     def __repr__(self):
9         return f"${self.dollars}"
10
11 wallet1 = Money(20)
12 wallet2 = Money(15)
13 combined = wallet1 + wallet2  # calls wallet1.__add__(wallet2)
14
15 print(combined)  # $35

```

`wallet1 + wallet2` is translated by Python into `wallet1.__add__(wallet2)`, following the exact same automatic-translation pattern as every magic method in this lesson; `__add__` decides what adding two `Money` objects together should actually produce, here a brand new `Money` holding the combined total, exactly mirroring how `+` already behaves for the built-in numeric types you’ve used since Lesson 1.4. `__sub__`, `__mul__`, and the rest of the arithmetic operators follow this identical pattern, one magic method per operator, only worth defining on a class where that operator genuinely has a sensible, unambiguous meaning; overloading `+` to mean something a reader wouldn’t naturally expect is a common, real source of confusing code.

A Reference Table of Common Magic Methods

Magic method	Triggered by	Purpose
<code>__init__</code>	<code>ClassName(...)</code>	Set up a new object’s initial state (Lesson 2.1)
<code>__str__</code>	<code>print(obj)</code> , <code>str(obj)</code>	Human-readable display
<code>__repr__</code>	interactive display, debugging	Unambiguous, developer-facing display
<code>__eq__</code>	<code>obj1 == obj2</code>	Define what equality means
<code>__lt__</code>	<code>obj1 < obj2</code> , <code>.sort()</code> , <code>sorted()</code>	Define ordering
<code>__len__</code>	<code>len(obj)</code>	Report a length
<code>__getitem__</code>	<code>obj[index]</code> , <code>for x in obj</code>	Support indexing and iteration
<code>__add__</code>	<code>obj1 + obj2</code>	Define what <code>+</code> means

Common Mistakes to Avoid

- **Calling a magic method directly, like `my_obj.__str__()`.** It works, technically, but it’s never how real code should call it; use the normal operation (`str(my_obj)`, `print(my_obj)`) and let Python translate it automatically.
- **Defining `__eq__` but forgetting other might not be the same type.** Comparing `self.x` against an other that isn’t even a `Point` can raise an `AttributeError`; production code typically checks `isinstance(other, Point)` first, though this course’s exercises keep it simple.
- **Confusing `__str__` and `__repr__`’s purposes.** `__str__` is for a human-friendly display; `__repr__` is for unambiguous debugging output, and serves as the fallback when `__str__` isn’t defined at all.
- **Overloading an operator to do something surprising.** Defining `__add__` to do something unrelated to genuine addition (deleting data, for instance) violates what any reader would reasonably expect from seeing `+`, and makes code far harder to trust at a glance.
- **Forgetting that defining `__lt__` alone is enough for sorting.** You don’t need to define every comparison magic method just to make `.sort()` or `sorted()` work; `__lt__` alone is sufficient for those specifically.

Lesson 2.6 Project: Custom Vector Class

Write a program called `vector.py` that:

1. Defines a class `Vector2D` with an `__init__` accepting `x` and `y`.

2. Implements `__repr__` returning something like `"Vector2D(3, 4)"`.
3. Implements `__eq__`, considering two vectors equal when both coordinates match.
4. Implements `__add__`, returning a new `Vector2D` with each coordinate summed, and `__sub__` doing the equivalent for subtraction.
5. Implements `__len__`, returning the vector's magnitude rounded to the nearest whole number (using `math.sqrt(self.x ** 2 + self.y ** 2)` from Lesson 1.17, and `round()` from Lesson 1.3), so `len()` gives a sensible, if approximate, "size" for the vector, acknowledging that `__len__` technically must return an `int`.
6. Creates a few `Vector2D` objects, demonstrates addition and subtraction with `+` and `-`, confirms equality checking works correctly, and prints each result using the `__repr__` you defined.

Lesson Summary: Quick Reference

- A magic (dunder) method is called automatically by Python in response to an operation or built-in function, never called directly by name in normal code
- `__str__` controls `print()/str()`; `__repr__` controls unambiguous, developer-facing display, and is the fallback if `__str__` is missing
- `__eq__` defines `==`; `__lt__` (and friends) define ordering and are enough alone to make `.sort()/sorted()` work
- `__len__` supports `len()`; `__getitem__` supports `obj[index]` and, on its own, basic for iteration
- `__add__` and its siblings (`__sub__`, `__mul__`, ...) let you define what arithmetic operators mean for your own class

Lesson 2.7: Decorators

Why This Matters

You've been using decorators since Lesson 2.3: `@classmethod`, `@staticmethod`, and `@property` from Lesson 2.5 are all decorators, and you've treated each one as a built-in label without yet seeing what a decorator actually is underneath. This lesson closes that gap. A decorator is simply a function that wraps another function, and once you can write your own, you gain a genuinely powerful, reusable way to add behavior, logging, timing, access checks, retry logic, to any function without changing that function's own code at all.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain precisely what `@decorator_name` above a function actually does
- Write your own decorator that wraps a function and adds behavior before or after it runs
- Preserve a decorated function's name and docstring using `functools.wraps`
- Write a decorator that accepts its own arguments

Recalling: Functions Are Values

Lesson 1.11 established that a function is a value in Python, just like a number or a string: it can be assigned to a variable, passed to another function, and, critically for this lesson, returned from another function. Decorators lean entirely on this one idea, so it's worth a quick, concrete reminder before building on it:

```
1 def shout(text):
2     return text.upper() + "!"
3
4 def apply_twice(func, value):
5     return func(func(value))
6
7 print(apply_twice(shout, "hi")) # HI!!
```

`apply_twice` takes a function, `func`, as an ordinary argument, and calls it twice, feeding its own output back into itself; nothing here is new syntax, just Lesson 1.11's "a function is a value" idea, used directly. A decorator is built from exactly this same capability, functions accepting and returning other functions, just organized into a specific, reusable pattern.

Writing Your First Decorator

A decorator is a function that takes a function as its argument, defines a new function that wraps it, adding behavior before or after the original runs, and returns that new, wrapping function:

```
1 def announce(func):
2     def wrapper():
3         print(f>About to call {func.__name__}...")
4         func()
5         print(f"Finished calling {func.__name__}.")
6     return wrapper
7
8 def greet():
9     print("Hello!")
10
```

```

11 greet = announce(greet) # manually wrapping greet with announce
12 greet()
13 # About to call greet...
14 # Hello!
15 # Finished calling greet.

```

Trace through this precisely, since the mechanics matter more here than in almost anything else this course has covered. `announce(greet)` is called with the original `greet` function as its argument, `func`. Inside `announce`, a brand new function, `wrapper`, is defined, one that prints a message, then calls the original `func`, then prints another message; `wrapper` is what `announce` returns. `greet = announce(greet)` reassigns the name `greet` to point at this new `wrapper` function instead of the original one, exactly the reassignment idea from Lesson 1.3, just now happening to a function value rather than a number. From that line onward, calling `greet()` actually calls `wrapper()`, which calls the original function (still reachable inside `wrapper` as `func`, thanks to the closure Lesson 3's advanced material explores further) along with the extra printed messages surrounding it.

The @ Syntax: Sugar for the Same Thing

Writing `greet = announce(greet)` by hand works, but Python provides a cleaner, equivalent shorthand: placing `@decorator_name` directly above a function definition applies the decorator immediately, automatically:

```

1 def announce(func):
2     def wrapper():
3         print(f>About to call {func.__name__}...")
4         func()
5         print(f"Finished calling {func.__name__}.")
6     return wrapper
7
8 @announce
9 def greet():
10    print("Hello!")
11
12 greet()
13 # About to call greet...
14 # Hello!
15 # Finished calling greet.

```

`@announce` directly above `def greet():` is exactly, precisely equivalent to writing `def greet(): ...` followed by `greet = announce(greet)` immediately afterward; it's purely a more readable way of writing the same reassignment, applied automatically the moment the function is defined. This is the entire mechanism behind `@classmethod`, `@staticmethod`, and `@property` from earlier lessons: each is a function (or, for `@property`, a class behaving similarly) that takes the method defined underneath it and returns something that changes how that method behaves, exactly the pattern `announce` demonstrates here in miniature.

Decorating Functions That Take Arguments

`announce`'s `wrapper` only worked for functions taking no arguments at all; a genuinely reusable decorator needs to handle any function, whatever arguments it happens to take. `*args` and `**kwargs` from Lesson 1.11 solve this precisely:

```

1 def announce(func):
2     def wrapper(*args, **kwargs):
3         print(f>About to call {func.__name__}...")
4         result = func(*args, **kwargs)

```

```

5         print(f"Finished calling {func.__name__}.")
6         return result
7     return wrapper
8
9 @announce
10 def add(a, b):
11     return a + b
12
13 total = add(3, 4)
14 # About to call add...
15 # Finished calling add.
16 print(total) # 7

```

`wrapper(*args, **kwargs)` accepts absolutely any combination of positional and keyword arguments, exactly the way `*args/**kwargs` were introduced in Lesson 1.11, and `func(*args, **kwargs)` forwards every single one of them straight through to the original function, unpacking them back out, following the exact same unpacking syntax from that same lesson. Capturing and returning `result` matters too: without it, a decorated function that's meant to return a value, like `add`, would silently return `None` instead, since `wrapper` itself would never hand anything back, the exact same “forgot to return” mistake Lesson 1.11 first warned about, just one level removed inside a decorator.

Try It Yourself 2.7a: Write a decorator `shout_result` that wraps a function, calls it normally, and returns its result converted to uppercase using `.upper()` from Lesson 1.7 (assume the wrapped function always returns a string). Apply it with `@shout_result` to a function that returns a greeting, and confirm the final printed result is fully capitalized.

Preserving Identity: `functools.wraps`

There's a subtle problem with every decorator written so far: the wrapping function replaces the original entirely, which means the original function's name and docstring, from Lesson 1.12, get quietly lost.

```

1 @announce
2 def add(a, b):
3     """Return the sum of a and b."""
4     return a + b
5
6 print(add.__name__) # wrapper - NOT "add"! The original identity is lost.
7 print(add.__doc__) # None - the original docstring is gone too

```

`add` now genuinely is `wrapper`, as far as Python is concerned, since that's literally the function object `@announce` left behind; anything inspecting `add.__name__` or `add.__doc__`, including `help()` from Lesson 1.12, sees `wrapper`'s details, not `add`'s own. The standard library's `functools` module (briefly previewed as `functools.partial/lru_cache` territory in Phase 3) provides `wraps`, itself a decorator, specifically to fix this:

```

1 from functools import wraps
2
3 def announce(func):
4     @wraps(func)
5     def wrapper(*args, **kwargs):
6         print(f>About to call {func.__name__}...")
7         result = func(*args, **kwargs)
8         print(f"Finished calling {func.__name__}.")
9         return result
10    return wrapper

```

```

11
12 @announce
13 def add(a, b):
14     """Return the sum of a and b."""
15     return a + b
16
17 print(add.__name__) # add - correct now
18 print(add.__doc__) # Return the sum of a and b.

```

`@wraps(func)` directly above the inner wrapper definition copies `func`’s `__name__`, `__doc__`, and a few other identifying details onto wrapper, so that the decorated function still identifies itself correctly to anything inspecting it, even though wrapper is still, mechanically, the function actually running. This is worth treating as a standard habit, not an optional extra: essentially every real decorator you’ll encounter in professional Python code includes `@wraps(func)`, and omitting it is a genuine, if subtle, bug.

Decorators That Take Their Own Arguments

Sometimes a decorator itself needs configuring, not just the function it wraps; `@repeat(3)`, for instance, might mean “call the wrapped function three times.” This needs one additional layer of nesting, a function that takes the decorator’s own argument and returns the actual decorator:

```

1 from functools import wraps
2
3 def repeat(times):
4     def decorator(func):
5         @wraps(func)
6         def wrapper(*args, **kwargs):
7             for _ in range(times):
8                 func(*args, **kwargs)
9             return wrapper
10    return decorator
11
12 @repeat(3)
13 def greet(name):
14     print(f"Hello, {name}!")
15
16 greet("Ada")
17 # Hello, Ada!
18 # Hello, Ada!
19 # Hello, Ada!

```

Read this from the outside in. `repeat(3)` is called first, immediately, with `times` set to 3; it returns `decorator`, a genuine decorator of the exact shape you’ve seen throughout this lesson, one that happens to remember `times` through the closure. `@repeat(3)` above `greet` then applies that returned decorator to `greet`, exactly as `@announce` applied `announce` directly, earlier in this lesson; the only difference is this extra outer layer that exists purely to accept `repeat`’s own configuration argument, 3, before building the actual decorator around it. `for _ in range(times):` from Lesson 1.6 uses `_` as the loop variable’s name, a common convention meaning “I need to repeat this a certain number of times, but I don’t actually need the loop counter’s value itself.”

Try It Yourself 2.7b: Write a decorator factory `min_args(n)` that returns a decorator which checks, before calling the wrapped function, whether it received at least `n` positional arguments (check `len(args)` inside `wrapper`), raising a `ValueError` from Lesson 1.14 if not, and otherwise calling and returning the function normally. Apply `@min_args(2)` to a small function and confirm it raises correctly when called with too few arguments.

Common Mistakes to Avoid

- **Forgetting `*args`, `**kwargs` in the wrapper.** A wrapper defined with no parameters at all can only decorate functions that also take no arguments; nearly every real decorator needs `*args`, `**kwargs` to stay genuinely general-purpose.
- **Forgetting to return result from the wrapper.** A decorated function that's supposed to return a value silently returns `None` instead, if the wrapper calls `func(...)` but never returns what that call produced.
- **Omitting `@wraps(func)`.** The decorated function's `__name__` and `__doc__` quietly become the wrapper's instead of the original's, which can be genuinely confusing while debugging or reading documentation later.
- **Confusing a decorator with arguments and one without.** `@repeat(3)` needs the extra outer function layer (a function returning a decorator); trying to reuse a plain, two-level decorator's structure for one that takes its own arguments results in a `TypeError` when it's applied.
- **Applying a decorator that expects a specific kind of function to the wrong kind.** A decorator written assuming the wrapped function always returns a string (like `shout_result` from this lesson's Try It Yourself) will fail if applied to a function returning something else, like an `int`; decorators aren't automatically as general-purpose as they might look.

Lesson 2.7 Project: Function Timing and Logging Toolkit

Write a program called `decorator_toolkit.py` that:

1. Writes a decorator `log_call` that prints the wrapped function's name and the arguments it was called with (using `args` and `kwargs` directly) before calling it, then returns its result, using `@wraps(func)` correctly.
2. Writes a decorator `timer` that uses `time.perf_counter()` (from the standard library's `time` module; call it once before and once after the wrapped function runs, and print the difference) to print how long the wrapped function took to execute, then returns its result.
3. Writes a decorator factory `retry(times)` that calls the wrapped function up to `times` times if it raises an exception, using `try/except` from Lesson 1.14, printing a message on each failed attempt, and re-raising the final exception if every attempt fails.
4. Applies at least two of these decorators, stacked together (`@log_call` and `@timer` both above the same function, one directly under the other), to a small sample function, and calls it to confirm both decorators' output appears, in the order you'd expect from how they're stacked.

This project deliberately builds three genuinely useful, realistic decorators, the kind that show up constantly in real Python codebases.

Lesson Summary: Quick Reference

- A decorator is a function that takes a function, defines a wrapping function around it, and returns that wrapper
- `@decorator` above a `def` is shorthand for `func = decorator(func)`, applied automatically
- A wrapper should accept `*args`, `**kwargs` and forward them to the original function, returning its result
- `@wraps(func)` from `functools` preserves the original function's `__name__` and `__doc__` on the wrapper
- A decorator that takes its own arguments needs one extra layer: a function accepting that argument and returning the actual decorator

Lesson 2.8: Abstract Base Classes

Why This Matters

Lesson 2.4 showed `Dog` and `Cat` each overriding `make_sound`, relying on every subclass remembering to actually define it. Nothing enforced that; a forgetful subclass that skipped `make_sound` entirely would simply inherit `Animal`'s version, or crash later with an `AttributeError` if `Animal` never defined one at all, and either way, the mistake wouldn't surface until something eventually tried to call it. An abstract base class lets you write a parent class that genuinely enforces a contract: any subclass must implement specific methods, or Python refuses to let you create it at all.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what an abstract base class is and why it enforces a contract rather than merely suggesting one
- Define an abstract base class using the `abc` module and `@abstractmethod`
- Explain why an abstract base class can't be instantiated directly
- Design a small hierarchy of concrete subclasses that each satisfy the same abstract contract

The Problem: An Unenforced Convention

Revisit the `Animal` hierarchy from Lesson 2.4, but imagine a genuinely large codebase, with many people adding new kinds of animal over time:

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5     def make_sound(self):
6         pass # intended as a placeholder every subclass should override
7
8 class Fish(Animal):
9     pass # oops - forgot to override make_sound
10
11 nemo = Fish("Nemo")
12 nemo.make_sound() # runs Animal's version, does nothing, no error, no warning
```

Nothing about this is technically broken, in the sense that Python raises no error at all; `Fish` simply inherits `Animal`'s `make_sound`, which does nothing, and the mistake, forgetting to override it, is completely silent. In a small program, this might get caught quickly; in a larger one, built by more than one person, a silently-missing override like this can sit undetected for a long time. Comments and naming conventions (like a docstring on `Animal` saying “subclasses must override this”) can document the intent, but nothing actually enforces it, the exact same “convention, not enforcement” gap Lesson 2.5 raised about single-underscore attribute names.

Defining an Abstract Base Class

The `abc` module (“abstract base classes”), part of the standard library, provides exactly the tool for turning that unenforced convention into a real, checked requirement. A class becomes an abstract base class by inheriting from `ABC`, and a method becomes required by decorating it with `@abstractmethod`:

```
1 from abc import ABC, abstractmethod
2
```

```
3 class Animal(ABC):
4     def __init__(self, name):
5         self.name = name
6
7     @abstractmethod
8     def make_sound(self):
9         pass
```

`class Animal(ABC):` makes `Animal` an abstract base class, by inheriting from `ABC`, exactly the same inheritance syntax from Lesson 2.4, just inheriting from a special class the standard library provides for this purpose. `@abstractmethod` directly above `make_sound` marks that specific method as required: any concrete (nonabstract) subclass must provide its own implementation, or Python won't allow that subclass to be instantiated at all. The body of an abstract method, here just `pass`, is essentially irrelevant, since it's never actually meant to run; `@abstractmethod` exists purely to declare "every subclass must define this," not to provide real behavior itself.

You Cannot Instantiate an Abstract Base Class Directly

This is the entire enforcement mechanism, made concrete:

```
1 animal = Animal("Generic")
2 # TypeError: Can't instantiate abstract class Animal with abstract method make_sound
```

Trying to create an `Animal` directly fails immediately, with a clear, specific `TypeError`, precisely because `Animal` has an unimplemented `@abstractmethod`. This is a deliberate, meaningful restriction: `Animal` here isn't meant to represent any actual animal on its own, only a common shape that real, specific animals must all follow; `ABC` and `@abstractmethod` together turn that intent into something Python actively checks, rather than something merely implied by a docstring or comment.

Concrete Subclasses Must Implement Every Abstract Method

A subclass that inherits from an abstract base class is called concrete once it implements every method that class marked as abstract; only then can it actually be instantiated:

```
1 class Fish(Animal):
2     pass # still hasn't implemented make_sound
3
4 nemo = Fish("Nemo")
5 # TypeError: Can't instantiate abstract class Fish with abstract method make_sound
```

The exact same enforcement extends automatically to subclasses: `Fish` inherited the unfulfilled `@abstractmethod` requirement from `Animal`, and until `Fish` actually defines its own `make_sound`, it remains just as impossible to instantiate as `Animal` itself was. This is precisely the guarantee an abstract base class provides, and precisely what the earlier, unenforced version of this example was missing entirely: the mistake of forgetting to override `make_sound` is caught immediately, the moment you try to create the object, rather than staying silent until something eventually calls the missing method.

```
1 class Fish(Animal):
2     def make_sound(self):
3         print(f"{self.name} blows bubbles.")
4
5 class Dog(Animal):
6     def make_sound(self):
```

```

7         print(f"{self.name} says Woof!")
8
9 nemo = Fish("Nemo") # works now - make_sound is implemented
10 rex = Dog("Rex")   # works too
11
12 nemo.make_sound() # Nemo blows bubbles.
13 rex.make_sound()  # Rex says Woof!

```

Both `Fish` and `Dog` are now genuinely concrete, since each provides its own `make_sound`, satisfying `Animal`'s contract; every regular inheritance rule from Lesson 2.4, overriding, `super()`, `isinstance()`, still applies completely normally on top of this, since an abstract base class is still an ordinary parent class in every other respect, just one with this extra, enforced requirement layered on.

Try It Yourself 2.8a: Define an abstract base class `Shape(ABC)` with an abstract method `area(self)`. Write two concrete subclasses, `Square` (storing a side length) and `Circle` (storing a radius), each implementing `area()` correctly. Confirm that trying to instantiate `Shape()` directly raises a `TypeError`, while both subclasses instantiate and compute their area correctly.

A Class Can Require Several Abstract Methods

An abstract base class isn't limited to a single required method; every method marked `@abstractmethod` must be implemented before a subclass counts as concrete:

```

1 from abc import ABC, abstractmethod
2
3 class PaymentMethod(ABC):
4     @abstractmethod
5     def process_payment(self, amount):
6         pass
7
8     @abstractmethod
9     def refund(self, amount):
10        pass
11
12 class CreditCard(PaymentMethod):
13     def process_payment(self, amount):
14         print(f"Charging ${amount} to credit card.")
15
16     def refund(self, amount):
17         print(f"Refunding ${amount} to credit card.")
18
19 card = CreditCard()
20 card.process_payment(50) # Charging $50 to credit card.

```

`CreditCard` only becomes concrete, and only becomes instantiable, once it provides implementations for both `process_payment` and `refund`; leaving either one unimplemented would raise the same `TypeError` seen earlier, naming whichever abstract methods are still missing. This is exactly the pattern real, professional Python code uses to define a genuine interface, a guaranteed set of methods every concrete implementation must provide, whether the underlying implementation is a credit card, a gift card, or a bank transfer; calling code that only ever interacts through `PaymentMethod`'s methods can trust that `process_payment` and `refund` will always exist, on absolutely any concrete subclass, without needing to check first.

Abstract Base Classes Can Still Have Regular Methods

An abstract base class isn't limited to only abstract methods; it can define completely ordinary, already-implemented methods too, shared normally by every subclass, exactly like any other inherited method from Lesson 2.4:

```

1 from abc import ABC, abstractmethod
2
3 class Animal(ABC):
4     def __init__(self, name):
5         self.name = name
6
7     @abstractmethod
8     def make_sound(self):
9         pass
10
11     def introduce(self): # NOT abstract - shared, ready-to-use behavior
12         print(f"I am {self.name}.")
13         self.make_sound()
14
15 class Dog(Animal):
16     def make_sound(self):
17         print("Woof!")
18
19 rex = Dog("Rex")
20 rex.introduce()
21 # I am Rex.
22 # Woof!

```

`introduce` is a completely ordinary method, inherited normally by `Dog` without needing to be overridden at all, and it even calls `self.make_sound()` internally, the exact one-method-calling-another pattern from Lesson 2.2, trusting that whatever concrete subclass `self` actually is, `make_sound` is guaranteed to exist and be properly implemented, precisely because `Animal` being an abstract base class enforces that guarantee. This combination, some methods abstract and required, others already fully implemented and shared, is exactly how a well-designed abstract base class typically looks: it provides real, working shared behavior wherever it can, and only demands each subclass fill in the specific pieces that genuinely must vary.

Try It Yourself 2.8b: Add a non-abstract method `describe(self)` to your `Shape` abstract base class from the earlier Try It Yourself, which prints something like "This shape has an area of 12.5.", calling `self.area()` internally rather than recalculating anything itself. Confirm it works correctly, unmodified, on both your `Square` and `Circle` subclasses.

Common Mistakes to Avoid

- **Forgetting to inherit from ABC.** `@abstractmethod` alone, on a class that doesn't inherit from `ABC`, does nothing to actually enforce the requirement; Python will happily instantiate the class and let the missing method go unnoticed until it's called.
- **Believing an abstract base class can be instantiated if you just don't call the abstract method.** It doesn't matter whether the abstract method would ever actually be called; Python refuses instantiation the moment any `@abstractmethod` remains unimplemented, checked immediately at creation time, not at call time.
- **Forgetting to implement every abstract method in a subclass, not just some of them.** A subclass with two abstract methods to satisfy, but only implementing one, is still abstract itself, and still can't be instantiated; the error message names whichever ones remain missing.

- **Assuming pass inside an abstract method needs to be meaningful code.** It doesn't; the body of an abstract method is essentially never executed directly, since subclasses are required to override it entirely, so pass (or a short docstring describing the expected contract) is the normal, correct content.
- **Overusing abstract base classes for simple hierarchies that don't need a strictly enforced contract.** If every subclass reasonably shares a sensible default implementation, and nothing genuinely requires every single subclass to define its own version, plain inheritance from Lesson 2.4, without ABC, is often the simpler, more appropriate tool.

Lesson 2.8 Project: Notification System

Write a program called `notification_system.py` that:

1. Defines an abstract base class `Notifier(ABC)` with abstract methods `send(self, message)` and `get_cost(self)` (returning a per-message cost as a float).
2. Adds a non-abstract method `send_with_receipt(self, message)` to `Notifier` that calls `self.send(message)`, then prints a receipt line showing the cost from `self.get_cost()`, using both abstract methods without needing to know which concrete subclass it's actually running on.
3. Defines at least two concrete subclasses, for example `EmailNotifier` and `SMSNotifier`, each implementing `send` (printing something appropriate, like "Emailing: <message>") and `get_cost` (returning a different, sensible value for each).
4. Confirms, using `try/except TypeError`: from Lesson 1.14, that attempting to instantiate `Notifier()` directly fails as expected, printing a message that confirms it.
5. Creates one of each concrete notifier, and calls `send_with_receipt()` on each with a sample message, showing the shared method working correctly across genuinely different underlying implementations.

Lesson Summary: Quick Reference

- `from abc import ABC, abstractmethod` provides the tools for enforced, abstract base classes
- `class Name(ABC):` makes a class abstract; `@abstractmethod` above a method makes overriding it mandatory
- An abstract base class, or any subclass still missing a required override, cannot be instantiated; Python raises `TypeError` immediately
- A subclass becomes concrete, and instantiable, only once every inherited abstract method has been implemented
- An abstract base class can still define ordinary, fully-implemented, shared methods alongside its abstract ones

Lesson 2.9: Dataclasses

Why This Matters

Look back at every class this phase has built, and a pattern emerges: a great many of them exist mainly to hold a fixed set of named data, `Point`'s `x` and `y`, `Product`'s `name` and `price`, with `__init__` doing nothing more interesting than assigning each parameter to `self`, and Lesson 2.6 showing you had to write `__repr__` and `__eq__` by hand just to get sensible printing and comparison. That's a lot of boilerplate for something so mechanical. This closing lesson introduces `@dataclass`, which generates all of it for you automatically, letting a class built primarily to hold data say so directly, in a single line, while remaining a completely ordinary class in every other respect.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what boilerplate `@dataclass` eliminates, and why it still requires type hints
- Define a dataclass with default values, including safely defaulting a mutable field
- Understand what `@dataclass` generates automatically, and add methods of your own alongside it
- Create an immutable, hashable dataclass with `frozen=True`

The Boilerplate Problem, Made Concrete

Compare the effort behind a plain, ordinary class that simply holds three pieces of data, written the way every class so far in this phase has been:

```
1 class Product:
2     def __init__(self, name, price, quantity):
3         self.name = name
4         self.price = price
5         self.quantity = quantity
6
7     def __repr__(self):
8         return f"Product(name={self.name!r}, price={self.price!r}, quantity={self.
9             quantity!r})"
10
11     def __eq__(self, other):
12         return (self.name, self.price, self.quantity) == (other.name, other.price, other
13             .quantity)
```

Every line here is doing something genuinely necessary, following exactly what Lessons 2.1 and 2.6 taught, and yet none of it is actually interesting: it's entirely mechanical, the same shape any data-holding class needs, rewritten by hand every single time. `@dataclass` exists specifically to generate this exact, predictable shape for you:

```
1 from dataclasses import dataclass
2
3 @dataclass
4 class Product:
5     name: str
6     price: float
7     quantity: int
```

These two versions of `Product` behave identically: both support `Product("Pen", 1.50, 10)`, both print a genuinely useful representation, both compare correctly with `==`. The second version simply doesn't require you to type out the mechanical part yourself. It's worth being precise about what "identically" means here: `@dataclass` doesn't do anything mysterious or unavailable to you otherwise; it generates real Python methods, an `__init__` that assigns each field to `self`, a `__repr__` built from the field names and values, an `__eq__` comparing every field, and attaches them to the class, exactly as if you'd typed them out by hand following Lesson 2.1 and Lesson 2.6's rules. Nothing about the resulting class is a different kind of thing; it's simply spared the repetition.

What `@dataclass` Actually Requires: Type Hints

Notice that `name: str`, `price: float`, and `quantity: int` are type hints, exactly the syntax Lesson 1.12 introduced for annotating variables. This isn't a coincidence, and it isn't optional: `@dataclass` uses these annotations to know which names to treat as fields, generating `__init__`, `__repr__`, and `__eq__` based on them. A line without a type hint is simply not picked up as a field at all.

```

1 product = Product("Pen", 1.50, 10)
2 print(product)           # Product(name='Pen', price=1.5, quantity=10)
3 print(product.name)      # Pen
4
5 pen2 = Product("Pen", 1.50, 10)
6 print(product == pen2)   # True - compares field values, exactly like Lesson 2.6's __eq__

```

Exactly as Lesson 1.12 stressed, these type hints are still just documentation as far as Python's runtime is concerned; `@dataclass` reads them to know what fields exist and in what order, but it does not actually validate that `price` is genuinely a `float` at runtime. Passing a string where `price` is annotated as `float` will not raise an error on its own; the annotation drives `dataclass`'s code generation, not runtime type checking.

Default Values

A field can have a default value, written exactly like a default parameter in a function, and following the identical rule from Lesson 1.11: every field with a default must come after every field without one.

```

1 @dataclass
2 class Product:
3     name: str
4     price: float
5     quantity: int = 0 # defaults to 0 if not provided
6
7 pen = Product("Pen", 1.50)
8 print(pen.quantity) # 0 - used the default
9
10 notebook = Product("Notebook", 4.99, 25)
11 print(notebook.quantity) # 25 - explicitly provided

```

This works exactly the way you'd expect from every default-parameter example since Lesson 1.11, since `@dataclass` is generating an `__init__` whose parameters follow that exact same shape, just written for you from the type-hinted fields above it.

The Mutable Default Trap, Revisited

Lesson 1.11 warned that a mutable value, like a list, can never be used as a plain default parameter value directly, since it gets created once, at definition time, and silently shared across every call. The exact same danger applies to `dataclass` fields, and `@dataclass` actively refuses to let you make this mistake by accident:

```

1 @dataclass
2 class Order:
3     items: list = [] # ValueError! dataclasses forbid a mutable default written
directly

```

Rather than silently creating the Lesson 1.11 bug for you, `@dataclass` raises a `ValueError` immediately, at class-definition time, refusing to proceed at all. The fix is `field()`, imported from the same `dataclasses` module, using its `default_factory` argument, a function called `fresh`, with no arguments, every time a new object is created without its own value supplied:

```

1 from dataclasses import dataclass, field
2
3 @dataclass
4 class Order:
5     order_id: str
6     items: list = field(default_factory=list)
7
8 order1 = Order("A100")
9 order2 = Order("A101")
10
11 order1.items.append("Pen")
12 print(order1.items) # ['Pen']
13 print(order2.items) # [] - genuinely independent, unlike Lesson 1.11's broken version

```

`default_factory=list` means “call `list()` fresh, producing a brand new empty list, every single time an `Order` is created without its own `items`,” exactly solving the aliasing problem from Lesson 1.11 (and, underneath it, the same aliasing idea Lesson 1.8 first introduced with plain lists) in a way `@dataclass` enforces you can’t get wrong.

Try It Yourself 2.9a: Define a dataclass `Student` with fields `name: str`, `grade: int`, and `courses: list` defaulting to an empty list via `field(default_factory=list)`. Create two `Student` objects, append a different course to each one’s `courses`, and confirm they remain independent, exactly like `Order` above.

Dataclasses Are Still Ordinary Classes

Everything from earlier in this phase still applies fully to a dataclass; `@dataclass` only generates a handful of specific magic methods, it doesn’t replace the class system itself. You can add your own methods, exactly like any class from Lesson 2.2:

```

1 @dataclass
2 class Product:
3     name: str
4     price: float
5     quantity: int = 0
6
7     def total_value(self):
8         return self.price * self.quantity
9
10 pen = Product("Pen", 1.50, 10)
11 print(pen.total_value()) # 15.0

```

`total_value` is a completely ordinary instance method, written exactly the way Lesson 2.2 taught, sitting comfortably alongside the fields `@dataclass` turned into `__init__`, `__repr__`, and `__eq__`. A dataclass can inherit and

be inherited from, following every rule from Lesson 2.4; it can define properties with `@property` from Lesson 2.5; it can even be an abstract base class. `@dataclass` is best understood as a targeted shortcut for the specific, common case of a class that's substantially about holding typed data, not a separate, different kind of class entirely.

Immutability: `frozen=True`

Passing `frozen=True` to the decorator makes every field genuinely unassignable after creation, turning the whole object immutable, the same idea Lesson 1.8 introduced for tuples, now available for a class you design yourself:

```
1 @dataclass(frozen=True)
2 class Point:
3     x: float
4     y: float
5
6 p = Point(3, 4)
7 print(p.x) # 3
8
9 p.x = 10 # FrozenInstanceError - cannot assign to a field of a frozen dataclass
```

This is worth reaching for whenever an object genuinely represents a fixed, unchanging value, a coordinate, an RGB color, a configuration snapshot, precisely the same reasoning Lesson 1.8 gave for choosing a tuple over a list. A useful, practical side effect of `frozen=True` specifically: a frozen dataclass automatically becomes hashable, meaning it can be used as a dictionary key or placed inside a set, from Lesson 1.9, something a normal, mutable object cannot safely do at all.

Automatic Ordering: `order=True`

Recall from Lesson 2.6 that defining `__lt__` by hand is enough to make `sorted()` work on a list of custom objects. `@dataclass` can generate every comparison method for you, comparing fields in the order they're declared, by passing `order=True`:

```
1 @dataclass(order=True)
2 class Product:
3     price: float
4     name: str
5
6 products = [Product(12.99, "Book"), Product(1.50, "Pen"), Product(8.00, "Mug")]
7 print(sorted(products))
8 # [Product(price=1.5, name='Pen'), Product(price=8.0, name='Mug'), Product(price=12.99,
9   name='Book')]
```

With `order=True`, comparisons like `<` compare each object's fields as a tuple, in declaration order, exactly the way tuples themselves compare element by element; here, that means comparing by `price` first (since it's declared first), which is precisely why `sorted()` produces the cheapest product first, with no `__lt__` written by hand at all.

Try It Yourself 2.9b: Redefine your `Student` dataclass from the earlier Try It Yourself with `order=True`, fields ordered as `grade: int` then `name: str`. Create several students with different grades, some sharing the same grade but different names, and confirm `sorted()` orders them by grade first, then by name among students sharing a grade.

Common Mistakes to Avoid

- **Forgetting type hints on a dataclass field.** `@dataclass` identifies fields specifically by their annotations; a line with no type hint at all is simply not treated as a field, and is easy to miss silently.
- **Writing a mutable default directly, like `items: list = []`.** `@dataclass` deliberately raises a `ValueError` rather than allowing this; use `field(default_factory=list)` instead.
- **Putting a field without a default before one that has one.** This raises a `TypeError` at class-definition time, the exact same ordering rule from Lesson 1.11's function parameters, just applied to dataclass fields.
- **Trying to modify a frozen dataclass's field.** `instance.field = new_value` raises `FrozenInstanceError` on a class decorated with `frozen=True`; create a new instance instead, exactly as reassigning a tuple in Lesson 1.8 required building a new one.
- **Believing `@dataclass` validates types at runtime.** It only reads the annotations to decide which fields exist and how to generate methods; it does not check, at runtime, that the value you actually passed matches the annotated type.

Lesson 2.9 Project: Game Character System (Phase 2 Capstone)

Write a program called `game_characters.py` that deliberately draws on ideas from across this entire phase:

1. Defines an abstract base class `Character(ABC)` (Lesson 2.8) with an abstract method `special_move(self)`, and ordinary attributes `name`, `health`, and `attack_power` set through a normal `__init__` (not a dataclass, since this class is abstract and needs custom logic).
2. Adds a `health` property (Lesson 2.5) with a setter that clamps the value so it can never be set below 0, no matter what value is assigned.
3. Defines at least two concrete subclasses (Lesson 2.4), such as `Warrior` and `Mage`, each implementing `special_move()` with its own printed behavior, and each implementing `__repr__` (Lesson 2.6) showing the character's name and current health.
4. Separately, defines a frozen dataclass (Lesson 2.9) `Item` with fields `name: str` and `power_bonus: int`, representing a piece of equipment, immutable once created.
5. Adds a method `equip(self, item)` to `Character` that increases `self.attack_power` by `item.power_bonus`, and prints a confirmation message.
6. Creates two or three characters of different concrete types, equips at least one with an `Item`, calls `special_move()` on each, and prints each character using its `__repr__`, demonstrating the whole hierarchy, the property, the abstract contract, and the dataclass working together in one connected program.

This project is a deliberate capstone for all of Phase 2: classes and objects, instance methods, inheritance and `super()`, encapsulation and properties, magic methods, and dataclasses, all working together.

Lesson Summary: Quick Reference

- `@dataclass` generates `__init__`, `__repr__`, and `__eq__` automatically, based on type-hinted class-level fields
- Fields with defaults must come after fields without, exactly like function parameters since Lesson 1.11
- Never write a mutable default directly; use `field(default_factory=list)` (or `dict`, `set`) instead
- A dataclass is still an ordinary class: it can have its own methods, properties, inheritance, and can be an abstract base class
- `frozen=True` makes fields unassignable after creation and makes instances hashable; `order=True` generates full comparison methods, ordered by field declaration order

Phase 2 Capstone Project: Library Management System

Just as Phase 1 closed with a project pulling together tools from across that entire phase, this capstone does the same for object-oriented Python. Every technique below was covered in one of the nine lessons you've just completed; nothing here is new. Building it is the clearest proof you'll have that classes, inheritance, properties, magic methods, decorators, abstract base classes, and dataclasses aren't nine separate tricks, but one coherent way of designing a program.

What You're Building

A command-line library system, `library_system.py`, that manages a catalog of different kinds of items (books, magazines, DVDs), tracks who has borrowed what, and enforces rules, like a maximum number of items any one member can borrow at once, entirely through the class design itself.

Capstone Requirements

1. **Abstract base class (Lesson 2.8).** Define `LibraryItem(ABC)` with abstract methods `check_out_period(self)` (returning the number of days that kind of item can be borrowed for) and `__repr__` (Lesson 2.6), plus a normal `__init__` storing `title` and a `_is_checked_out` flag, defaulting to `False`.
2. **Encapsulation (Lesson 2.5).** Expose `is_checked_out` as a read-only `@property`; the only way to change it should be through the methods in the next requirement, never by assigning to it directly from outside the class.
3. **Instance methods (Lesson 2.2).** Add `check_out(self)` and `return_item(self)` methods that update `_is_checked_out`, each printing a clear message, and each refusing to do anything (printing a warning instead) if the item is already in that state, for example trying to check out something already checked out.
4. **Concrete subclasses (Lesson 2.4).** Implement at least three subclasses of `LibraryItem`, for example `Book`, `Magazine`, and `DVD`, each with its own `checkout_period()` (books might allow 21 days, DVDs only 7) and its own `__repr__`.
5. **Magic methods (Lesson 2.6).** Give `LibraryItem` an `__eq__` comparing by title, and `__lt__` comparing by `checkout_period()`, so a list of mixed items can be sorted directly with `sorted()`.
6. **A frozen dataclass (Lesson 2.9).** Define `Member` as a dataclass with `name: str` and `member_id: str`, decorated with `frozen=True`, since a member's identity shouldn't change once created.
7. **A class attribute and class method (Lessons 2.1, 2.3).** On `LibraryItem`, track a class attribute `total_checked_out` across every item of every kind, incremented in `check_out()` and decremented in `return_item()`, with a `@classmethod` `get_total_checked_out(cls)` to report it.
8. **A decorator (Lesson 2.7).** Write a decorator `log_action` that wraps `check_out` and `return_item`, printing the action taken and the item's title before calling the original method, applied with `@log_action` directly above each.
9. **Putting it together.** Create a small catalog of mixed items and a couple of `Member` objects, check some items out, attempt to check out an already-checked-out item to confirm the guard works, return an item, and print `LibraryItem.get_total_checked_out()` after each change.

Stretch Goals

If the core system feels solid, extend it with tools from Phase 1: save the catalog to `library_catalog.json` between runs (Lesson 1.13), using `try/except` (Lesson 1.14) to handle a missing or corrupted file; or add a `due_date` attribute set with `datetime.date.today()` plus the item's `checkout_period()` as a `timedelta` (Lesson 1.17) whenever something is checked out.